

Programação Aplicada

Interfaces Gráficos

Listagens, Tabelas e Formatações

Marco Veloso
marco.veloso@estgoh.ipc.pt

Agenda

Interfaces Gráficas – Listagens e Tabelas

Elementos Gráficos, AWT e Applets

Interface Gráficos

Componentes de um Interface Gráfico

Gestão de Eventos

Contentores e Gestores de Posicionamento

Interacção com o utilizador

Caixas de diálogo e menus

Elementos Multimédia

Listagens e Tabelas

Formatações gráficas

Organização de dados e listagens

- **Tabelas;**
- **Listas.**

Organização de áreas (I)

- **Margens;**
- **Abas;**
- **Barras de deslocamento;**
- **Dimensionamento e Foco.**

Formatação de campos alfa-numéricos

- ***Spinners;***
- **Campos de texto formatados;**
- **Formatação de texto e números;**
- **Manipulação e formatação de datas.**

Organização de áreas (II)

- **Divisão de áreas;**
- **Árvores.**

Propriedades de uma janela

Interfaces Gráficas – Listagens e Tabelas

Principais propriedades das componentes gráficas

```
setSize(dimH,dimV); / setPreferredSize(new Dimension(dimH,dimV));

pack();           // if size not defined, use pack() that causes Window to be sized to
                  // fit the preferred size and layouts of its subcomponents

setResizable(true|false);
setTitle("Nome da Janela");
setVisible(true|false);
setEnabled(true|false);
setLocationRelativeTo(null);           // center according to monitor
setLocationRelativeTo(parent);         // center according to main app
setAlwaysOnTop(true);
toFront();
requestFocus(); / requestFocusInWindow();
setDefaultCloseOperation(WindowConstants.DO_NOTHING_ON_CLOSE);
setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
frame.getRootPane().setDefaultButton(JButton);
setIconImage(Toolkit.getDefaultToolkit().getImage("/images/image.png"));
```

Templates em JavaSwing e JavaFX

Interfaces Gráficas – Listagens e Tabelas

Como em outras linguagens de programação, em Java existem *templates* (modelos) que podem ser integrados nos projectos gráficos para personalizar a forma de apresentação (*look and feel*) das interfaces gráficas em JavaSwing e JavaFX

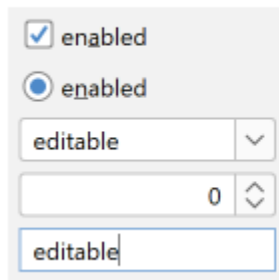
Estes *templates* são externos à biblioteca java, pelo que devem ser descarregados das páginas dos desenvolvedores e importados para o nosso projecto gráfico

Um *template* comum é o **FlatLaf - Flat Look and Feel**

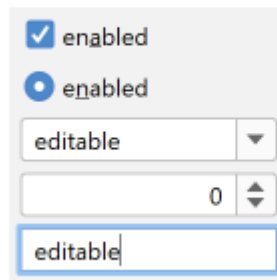
<https://www.formdev.com/flatlaf/>

```
import com.formdev.flatlaf.FlatDarculaLaf;  
FlatDarculaLaf.setup();  
// Código para criação da IU
```

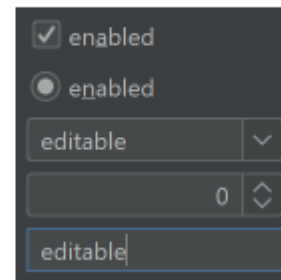
FlatLaf Light

A sample of the FlatLaf Light look and feel. It features a light gray background. The controls include a checked checkbox labeled 'enabled', a radio button labeled 'enabled', a text field with 'editable' and a dropdown arrow, a numeric spinner with '0', and a text field with 'editable'.

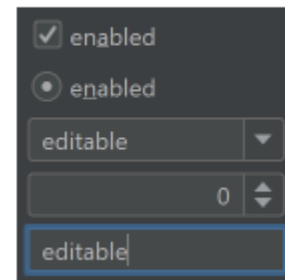
FlatLaf IntelliJ

A sample of the FlatLaf IntelliJ look and feel. It features a light gray background. The controls include a checked checkbox labeled 'enabled', a radio button labeled 'enabled', a text field with 'editable' and a dropdown arrow, a numeric spinner with '0', and a text field with 'editable'.

FlatLaf Dark

A sample of the FlatLaf Dark look and feel. It features a dark gray background. The controls include a checked checkbox labeled 'enabled', a radio button labeled 'enabled', a text field with 'editable' and a dropdown arrow, a numeric spinner with '0', and a text field with 'editable'.

FlatLaf Darcula

A sample of the FlatLaf Darcula look and feel. It features a dark gray background. The controls include a checked checkbox labeled 'enabled', a radio button labeled 'enabled', a text field with 'editable' and a dropdown arrow, a numeric spinner with '0', and a text field with 'editable'.

Organização de dados e listagens:

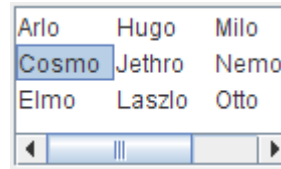
Listagens e Tabelas

Listas

Formatações: Listas

Interfaces Gráficas – Listagens e Tabelas

Uma **lista (JList)** apresenta um grupo de elementos, que podem ser apresentados numa ou mais colunas, podendo o utilizador seleccionar um ou mais elementos



A lista recebe como argumento uma tabela (*array*) com os elementos a disponibilizar

```
String [] data = {"PA", "BD", "ALGA"};  
JList list = new JList( data );  
add( list );
```

Para obter o elemento seleccionado pelo utilizador, invoca-se o método

getSelectedIndex()

que **devolve o índice (posição) do elemento seleccionado**, permitindo posteriormente o seu acesso na tabela original

```
System.out.println( data[ list.getSelectedIndex() ] );
```


Associado às listas existem duas propriedades relevantes:

- plano ou **orientação como os elementos são apresentados** (verticalmente ou horizontalmente) através do método

`setLayoutOrientation()`

- e **modo como podem ser selecionados**, através do método

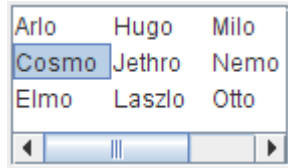
`setSelectionMode()`

```
String [] data = {"PA", "BD", "ALGA"};
JList list = new JList( data );
list.setSelectionMode(
    ListSelectionModel.SINGLE_INTERVAL_SELECTION);
list.setLayoutOrientation( JList.HORIZONTAL_WRAP );
```

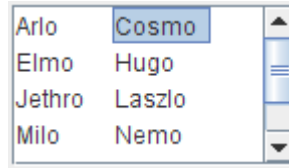
Formatações: Listas

Interfaces Gráficas – Listagens e Tabelas

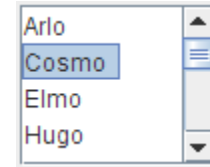
Existem várias orientações de **dispor os elementos** numa lista:



HORIZONTAL_WRAP



VERTICAL_WRAP



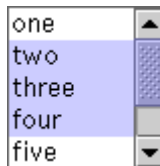
VERTICAL

Em termos de **selecção** pode ser restringida a apenas **um elemento** ou possibilitar uma **escolha múltipla**:

SINGLE_INTERVAL_SELECTION



SINGLE_SELECTION



MULTIPLE_INTERVAL_SELECTION

Caso a lista permita a **selecção simultânea de vários elementos** (**SINGLE_INTERVAL_SELECTION** ou **MULTIPLE_INTERVAL_SELECTION**) deve-se recorrer ao método

getSelectedIndices()

para obter **quais os elementos seleccionados pelo utilizador**, ao invés do método anterior

getSelectedIndex()

que apenas devolve o índice do primeiro elemento seleccionado

O método devolve uma **tabela de inteiros** (**int []**) com os índices dos elementos seleccionados

Formatações: Listas

Interfaces Gráficos – Listagens e Tabelas

Devido à quantidade de elementos que podem ser inseridos numa lista é comum **integra-las em painéis de deslocamento** (**JScrollPane**)

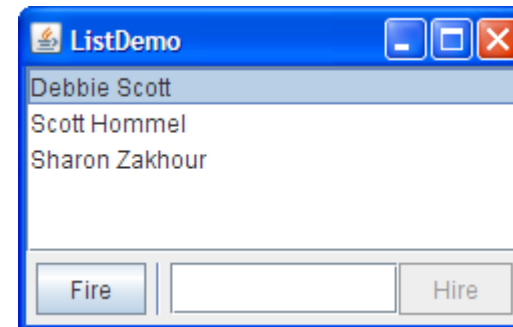
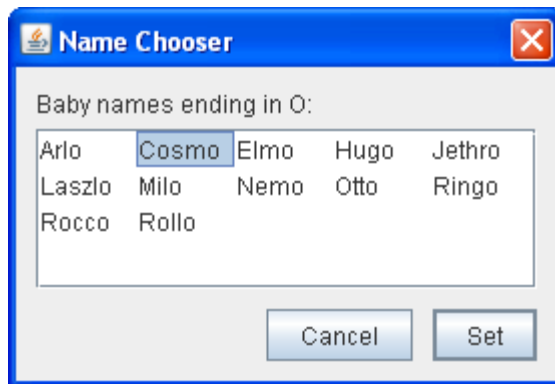
```
JList list = new JList( data ); //data has type Object[]
list.setSelectionMode(
    ListSelectionModel.SINGLE_INTERVAL_SELECTION );
list.setLayoutOrientation( JList.HORIZONTAL_WRAP );
list.setVisibleRowCount( -1 );
...
JScrollPane listScroller = new JScrollPane( list );
listScroller.setPreferredSize( new Dimension(250, 80) );
add( listScroller );
```

O método `setVisibleRowCount` define o **número de linhas ou colunas (dependendo da orientação) a apresentar antes de incluir uma barra de deslocamento** (*scroll pane*). O valor por defeito -1 permite que o compilador decida quando deve aplicar a barra de deslocamento

Para a selecção dos elementos é ainda possível recorrer a outros componentes como menus, tabelas, *combo boxes*, *radio buttons* ou *check boxes*

Formatações: Listas

Interfaces Gráficas – Listagens e Tabelas



Mais informações em

<http://docs.oracle.com/javase/tutorial/uiswing/components/list.html>

Tabelas

Formatações: Tabelas

Interfaces Gráficos – Listagens e Tabelas

As **tabelas** (**JTable**) permitem visualizar a informação de uma forma formatada



Uma tabela é **inicializada** com a instrução

```
JTable table = new JTable(data, columnNames)  
add(table) ;
```

que recebe como argumento uma **tabela** (ou Vector) '**data**' com os **dados a disponibilizar** e uma **tabela** (ou Vector) '**columnNames**' com o **nome das colunas**

É recomendável associar uma tabela a um **scroll pane** para **visualizar o cabeçalho** e poder **interagir** com os vários elementos:

```
JTable table = new JTable(data, columnNames)
JScrollPane scrollPane = new JScrollPane( table );
add( scrollPane );
```

Podemos definir o **tamanho por defeito** da tabela através do método

```
setPreferredSize()
```

que recebe uma dimensão (*Dimension*):

```
table.setPreferredSize( new Dimension(500, 70));
```

Caso o tamanho não seja definido manualmente, o compilador calculará qual a dimensão adequada de acordo com a *frame* activa

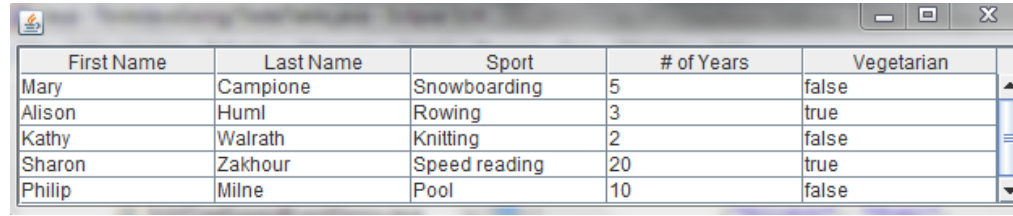
O método **setFillViewportHeight()** define que a tabela deverá (ou não) **preencher o espaço disponível** na *frame*

```
table.setFillViewportHeight( true );
```


Formatações: Tabelas

Interfaces Gráficos – Listagens e Tabelas

Simple implementação de uma tabela (com *scrollbar*):



First Name	Last Name	Sport	# of Years	Vegetarian
Mary	Campione	Snowboarding	5	false
Alison	Huml	Rowing	3	true
Kathy	Walrath	Knitting	2	false
Sharon	Zakhour	Speed reading	20	true
Philip	Milne	Pool	10	false

```
String[] columnNames = {"First Name",  
                           "Last Name",  
                           "Sport",  
                           "# of Years",  
                           "Vegetarian"};
```

```
Object[][] data = {  
    {"Mary",    "Campione", "Snowboarding",    new Integer(5),    new Boolean(false)},  
    {"Alison",  "Huml",     "Rowing",      new Integer(3),    new Boolean(true)},  
    {"Kathy",   "Walrath",  "Knitting",   new Integer(2),    new Boolean(false)},  
    {"Sharon",  "Zakhour",  "Speed reading", new Integer(20), new Boolean(true)},  
    {"Philip",  "Milne",    "Pool",      new Integer(10), new Boolean(false)},  
};
```

```
JTable table = new JTable( data, columnNames );  
table.setPreferredScrollableViewportSize( new Dimension(500, 70) );  
table.setFillsViewportHeight( true );
```

```
JScrollPane scrollPane = new JScrollPane( table );  
add( scrollPane );
```

Formatações: Tabelas

Interfaces Gráficos – Listagens e Tabelas

```
import java.util.Vector;
import java.awt.BorderLayout;
import javax.swing.JFrame; import javax.swing.JScrollPane; import javax.swing.JTable;

public class JTableCreatingByVector {
    public static void main(String args[]) {
        Vector<String> rowOne = new Vector<String>();
        rowOne.addElement("Row1-Column1");
        rowOne.addElement("Row1-Column2");
        rowOne.addElement("Row1-Column3");

        Vector<String> rowTwo = new Vector<String>();
        rowTwo.addElement("Row2-Column1");
        rowTwo.addElement("Row2-Column2");
        rowTwo.addElement("Row2-Column3");

        Vector<Vector> rowData = new Vector<Vector>();
        rowData.addElement( rowOne );
        rowData.addElement( rowTwo );

        Vector<String> columnNames = new Vector<String>();
        columnNames.addElement("Column One");
        columnNames.addElement("Column Two");
        columnNames.addElement("Column Three");

        JTable table = new JTable(rowData, columnNames);

        JScrollPane scrollPane = new JScrollPane( table );

        JFrame frame = new JFrame();
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        frame.add(scrollPane, BorderLayout.CENTER);
        frame.setSize(300, 150);
        frame.setVisible( true );
    }
}
```

Formatações: Tabelas

Interfaces Gráficos – Listagens e Tabelas

Uma vez criada uma tabela poderemos **obter as suas dimensões** (número de linhas e número de colunas) através dos métodos

`getRowCount()` e
`getColumnCount()`

Exemplo:

```
JTable table = new JTable(data, columnNames);  
int numRows = table.getRowCount();           // número de linhas  
int numCols = table.getColumnCount();         // número de colunas
```

Formatações: Tabelas

Interfaces Gráficos – Listagens e Tabelas

O método `getModel()` devolve os **dados da tabela** (na representação de um objecto `TableModel`)

```
TableModel model = table.getModel();
```

É assim possível **aceder ao valor de uma determinada posição da tabela**

```
getValueAt()
```

ou percorrer toda a tabela:

```
int numRows = table.getRowCount();           // número de linhas
int numCols = table.getColumnCount();         // número de colunas
TableModel model = table.getModel();       // dados da tabela

for (int i=0; i < numRows; i++) {
    System.out.print("    row " + i + ":");
    for (int j=0; j < numCols; j++) {
        System.out.print("  " + model.getValueAt(i, j));
    }
    System.out.println();
}
```

Formatações: Tabelas

Interfaces Gráficas – Listagens e Tabelas

Tal como nas listas (`JList`) é **possível controlar o tipo de acção e selecção** que o utilizador executa sobre a tabela, através do método

`setSelectionMode()`

que define uma das seguintes opções:

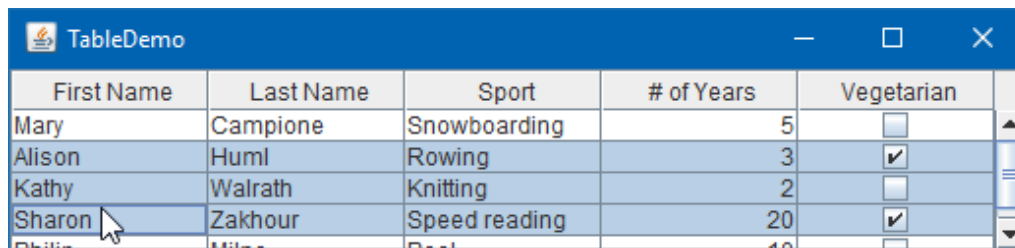
`MULTIPLE_INTERVAL_SELECTION`,

`SINGLE_INTERVAL_SELECTION`, e

`SINGLE_SELECTION`

Por exemplo:

```
table.setSelectionMode(  
    ListSelectionModel.SINGLE_INTERVAL_SELECTION );
```



First Name	Last Name	Sport	# of Years	Vegetarian
Mary	Campione	Snowboarding	5	☐
Alison	Huml	Rowing	3	☒
Kathy	Walrath	Knitting	2	☐
Sharon	Zakhour	Speed reading	20	☒
Philip	Wiles	Boat	10	☐

Sendo igualmente possível conhecer qual a **linha seleccionada** pelo utilizador, através do método `getSelectedRow()`

```
int index = table.getSelectedRow();
```

Caso a tabela permita a selecção simultânea de vários elementos (`SINGLE_INTERVAL_SELECTION` ou `MULTIPLE_INTERVAL_SELECTION`) deve-se recorrer ao método

```
getSelectedRows()
```

para obter **quais as linhas seleccionados pelo utilizador**, ao invés do método anterior `getSelectedRow()` que apenas devolve o índice da primeira linha seleccionada

O método devolve uma tabela de inteiros (`int []`) com os índices das linhas seleccionadas

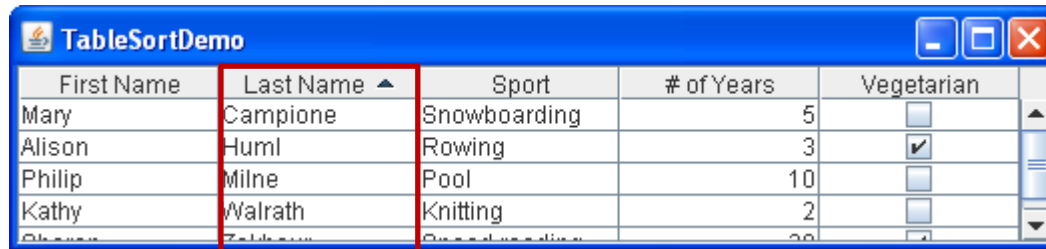
```
int [] indices = table.getSelectedRows();
```

Formatações: Tabelas

Interfaces Gráficos – Listagens e Tabelas

A **ordenação de valores** pode ser realizada invocando o método `setAutoCreateRowSorter(true)`

```
JTable table = new JTable(data, columnNames);  
table.setAutoCreateRowSorter( true );
```



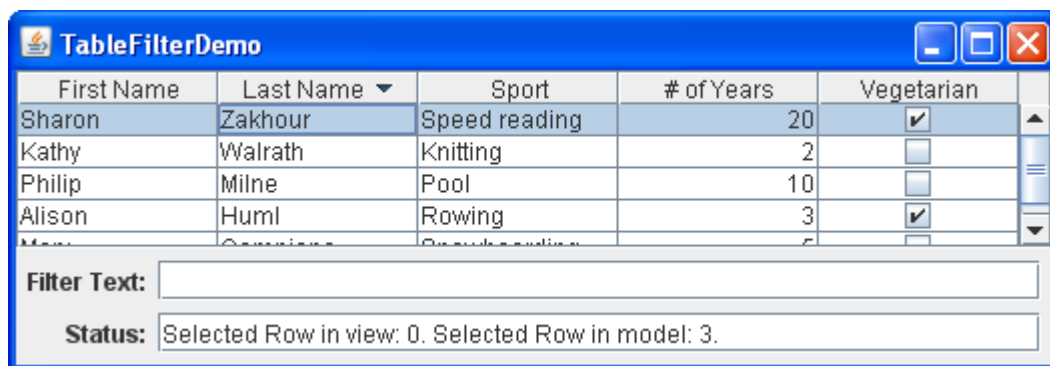
First Name	Last Name ▲	Sport	# of Years	Vegetarian
Mary	Campione	Snowboarding	5	☐
Alison	Huml	Rowing	3	☒
Philip	Milne	Pool	10	☐
Kathy	Walrath	Knitting	2	☐
Oliver	Zalazay	Speedboarding	20	☐

Para um maior controlo da ordenação dos dados podemos implementar uma instância da interface `TableRowSorter` que usa a classe `comparator`

Formatações: Tabelas

Interfaces Gráficos – Listagens e Tabelas

A interface **TableRowSorter** também permite a **filtragem dos valores** através da classe **RowFilter**, que implementa vários métodos de filtragem, como o **regexFilter()**



Formatações: Tabelas

Interfaces Gráficos – Listagens e Tabelas

É possível detectar e tratar mudanças introduzidas nos dados da tabela (eventos) através da interface TableModelListener (semelhante às interfaces de gestão de eventos `ActionListener`, `MouseListener` OU `KeyListener`):

```
import javax.swing.event.*;
import javax.swing.table.TableModel;

public class MyClass... implements TableModelListener {
    ...
    public MyClass() {
        ...
        JTable table = new JTable(data, columnNames);
        table.getModel().addTableModelListener(this);
        ...
    }

    public void tableChanged(TableModelEvent e) {
        int row          = e.getFirstRow(); // linha onde ocorreu a alteração
        int column        = e.getColumn();   // coluna onde ocorreu a alteração
        TableModel model  = (TableModel)e.getSource();
        String columnName = model.getColumnName(column);
        Object data       = model.getValueAt(row, column); // valor alterado

        ...
        // Acção sobre os dados alterados
    }
}
```

Formatações: Tabelas

Interfaces Gráficas – Listagens e Tabelas

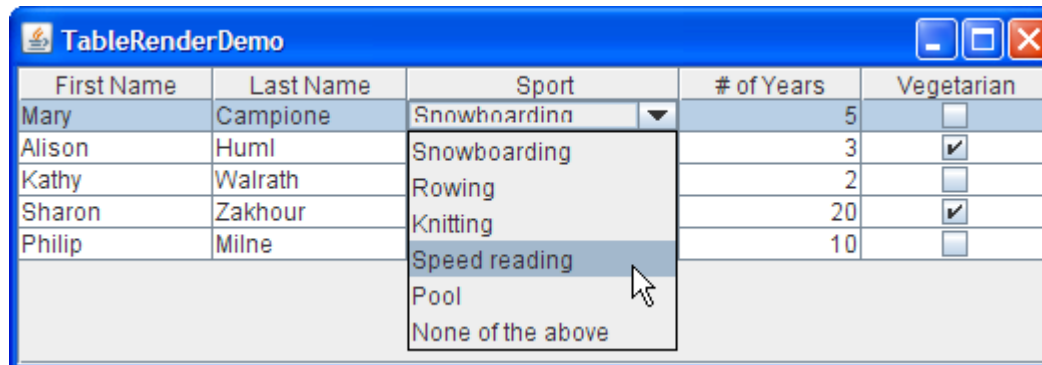
Para facilitar a edição/alteração de dados por parte do utilizador é possível usar **ComboBox** e outras **JComponents** em tabelas

Para o efeito obtemos a coluna da tabela que queremos modificar (**getColumn**) e sobre essa coluna alteramos o editor (**setCellEditor**) atribuindo uma **ComboBox**

```
JComboBox comboBox = new JComboBox();
comboBox.addItem("Snowboarding");
comboBox.addItem("Rowing");
comboBox.addItem("Knitting");
comboBox.addItem("Speed reading");
comboBox.addItem("Pool");
comboBox.addItem("None");
```

//...

```
JTable table = new JTable(data, columnNames);
TableColumn sportColumn = table.getColumnModel().getColumn(2);
sportColumn.setCellEditor( new DefaultCellEditor( comboBox ) );
```

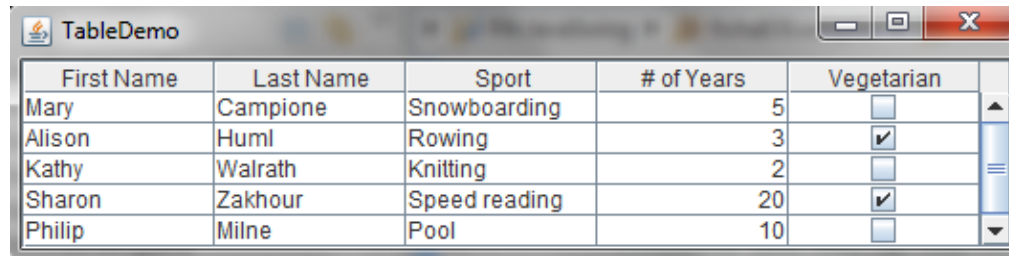


Formatações: Tabelas

Interfaces Gráficos – Listagens e Tabelas

É possível formatar as tabelas para melhorar a interacção com o utilizador.

Por exemplo, **transformar as colunas com valores do tipo *Boolean* (*true* / *false*) em *check boxes* e permitir a edição de campos**



First Name	Last Name	Sport	# of Years	Vegetarian
Mary	Campione	Snowboarding	5	☐
Alison	Huml	Rowing	3	☒
Kathy	Walrath	Knitting	2	☐
Sharon	Zakhour	Speed reading	20	☒
Philip	Milne	Pool	10	☐

Para o efeito usamos a classe **AbstractTableModel**, ou melhor, codificamos uma nova classe que implementa (herda) a classe **AbstractTableModel** que **representará a tabela** juntamente com as suas funcionalidades

```
class MyTableModel extends AbstractTableModel {  
    String[]    columnNames = { ... };  
    Object[][]  data = { ... }  
    //...  
}
```

Desta forma, a tabela passa a será inicializada pela nova classe

```
JTable table = new JTable( new MyTableModel() );
```

Formatações: Tabelas

Interfaces Gráficos – Listagens e Tabelas

Ao implementarmos a classe **AbstractTableModel** teremos que implementar os métodos:

```
getColumnCount(),           // retorna o número de colunas
getRowCount(),              // retorna o número de linhas
getColumnName(int) e        // retorna o nome de uma coluna
getValueAt(int, int)        // valor de uma determinada posição da tabela
```

Adicionalmente, para **transformar os valores do tipo Boolean em *checkboxes*** teremos que implementar o método

```
getColumnClass(int)         // Retorna o tipo de dados de um atributo para
                             // determinar quais os atributos do tipo Boolean
```

Para permitir que os **dados das células sejam editáveis** teremos que implementar os métodos

```
isCellEditable(int, int) e   // indica se uma célula pode ser editada
setValueAt(Object, int, int) // actualiza o valor editado graficamente
```

Formatações: Tabelas

Interfaces Gráficos – Listagens e Tabelas

```
class MyTableModel extends AbstractTableModel {
    private String[] columnNames;
    private Object[][] data;

    MyTableModel()
        columnName = {"First Name", "Last Name", "Sport", "# of Years", "Vegetarian"};
        data =
            { {"Mary", "Campione", "Snowboarding", new Integer(5), new Boolean(false)},
              {"Alison", "Huml", "Rowing", new Integer(3), new Boolean(true)},
              //...
            };

    public int getColumnCount() {
        return columnName.length;
    }

    public int getRowCount() {
        return data.length;
    }

    public String getColumnName(int col) {
        return columnName[col];
    }

    public Object getValueAt(int row, int col) {
        return data[row][col];
    }

    public Class getColumnClass(int c) {
        return getValueAt(0, c).getClass();
    }

    public boolean isCellEditable(int row, int col) {
        if (col < 2) {
            return false;
        } else {
            return true;
        }
    }

    public void setValueAt(Object value, int row, int col) {
        data[row][col] = value;
        fireTableCellUpdated(row, col);
    }
}
```

// retorna o número de colunas
// alternativa: data[0].length;

// retorna o número de linha

// retorna o nome de uma coluna

// retorna valor de uma determinada posição da tabela

// transforma os valores booleanos em check boxes
// devolve o tipo de valor para determinar qual é bool

// define que dados da tabela editáveis
// colunas 0 e 1 não são editáveis

// colunas 2 e seguintes são editáveis,

// actualiza os dados alterados graficamente
// na tabela
// método da interface AbstractTableModel que notifica
// os listeners da existência de alteração de dados

Formatações: Tabelas

Interfaces Gráficos – Listagens e Tabelas

```
public class TableDemo extends JFrame {
    public TableDemo() {
        Container cont = getContentPane();
        cont.setLayout( new FlowLayout() );

        JTable table = new JTable( new MyTableModel() );
        table.setPreferredScrollableViewportSize( new Dimension(500, 70)  );
        table.setFillViewportHeight( true );

        JScrollPane scrollPane = new JScrollPane( table );
        cont.add( scrollPane );
    }

    public static void main(String[] args) {
        JFrame frame = new TableDemo("TableDemo");
        frame.setTitle("TableDemo");
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);

        frame.pack();
        frame.setVisible(true);
    }
}

class MyTableModel extends AbstractTableModel { // definido na página anterior
    //...
}
```

Formatações: Tabelas

Interfaces Gráficos – Listagens e Tabelas

Para tornar a visualização mais intuitiva é comum **alterar os campos booleanos para que sejam apresentado uma *checkbox* ao invés da *string true/false*.**

Para o efeito, recorreremos à classe **AbstractTableModel**, como descrito anteriormente, permitindo maior controlo sobre a **JTable**

Alternativamente, para alterar a forma de apresentação dos campos **basta reescrever o método `getColumnClass(int column)`**, sem ter que implementar uma nova classe que herde de **AbstractTableModel**

```
JTable table = new JTable(...)
{
    @Override
    public Class getColumnClass(int column)
    {
        for (int row = 0; row < getRowCount(); row++)
        {
            Object obj = getValueAt(row, column);

            if (obj != null)
            {
                return obj.getClass();
            }
        }
        return Object.class;
    }
};
```

Formatações: Tabelas

Interfaces Gráficas – Listagens e Tabelas

É possível **actualizar o conteúdo da tabela** durante a execução, através da interface **DefaultTableModel**

```
String[][] data      = {...};           // dados, array ou vector
String[]  colName    = {"Val 1", "Val 2", "Val3"}; // nome das colunas, array ou vector

// Criação da JTable e definição dos parâmetros de configuração:
JTable table = new JTable();           // não introduz os dados na criação
table.setSelectionMode( ListSelectionMode.SINGLE_SELECTION );
JScrollPane scrollPane = new JScrollPane( table );
add( scrollPane );

DefaultTableModel tableModel = (DefaultTableModel) table.getModel();
tableModel.setColumnIdentifiers( colName ); // adiciona o nome das colunas

// inserir um registo
tableModel.addRow( data );           // adiciona os dados

// inserir um novo conjunto de registos (alternativamente pode-se iterar sobre o vector e
// inserir cada registo individualmente através do método addRow() )
tableModel.setDataVector( vectorData, vectorColum );

//força a tabela a redesenhar o seu conteúdo, verificando os novos dados.
tableModel.fireTableDataChanged();
table.setModel( tableModel );
```


Podemos **controlar o tamanho das diversas colunas** da tabela através do método **setPreferredWidth()**:

```
TableColumn column = null;

for (int i = 0; i < 5; i++) {
    column = table.getColumnModel().getColumn(i);
    if (i == 2) {
        //a terceira coluna tem um tamanho superior às restantes
        column.setPreferredWidth(100);
    } else {
        column.setPreferredWidth(50);
    }
}
```

Formatações: Tabelas

Interfaces Gráficos – Listagens e Tabelas

O **estilo do cabeçalho** da tabela é controlado através da classe **JTableHeader**, definindo a cor do texto (**setForeground**) e a cor de fundo (**setBackground**):

```
JTable table = new JTable(data, columnNames);  
table.setEnabled( true );  
  
JTableHeader tableHeader = table.getTableHeader();  
tableHeader.setForeground(new Color(0).white);  
tableHeader.setBackground(new Color(0).black);  
  
JScrollPane paneDis = new JScrollPane( table );
```

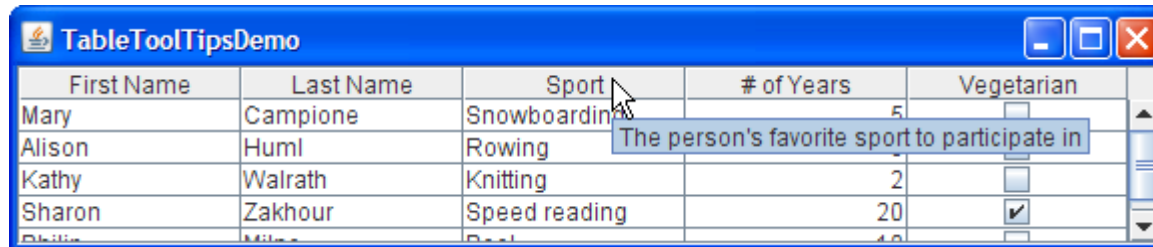
First Name	Last Name
Mary	Campione
Alison	Huml
Kathy	Walrath
Sharon	Zakhour
Philip	Milne

Formatações: Tabelas

Interfaces Gráficos – Listagens e Tabelas

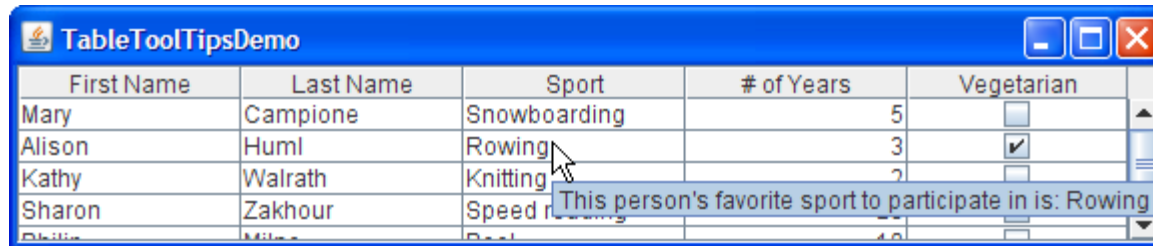
É possível **criar *tool tips*** para os cabeçalhos das colunas e para as diversas células:

```
protected JTableHeader createDefaultTableHeader() {  
    return new JTableHeader(columnModel) {  
        public String getToolTipText(MouseEvent e) {  
            String tip = null;  
            java.awt.Point p = e.getPoint();  
            int index      = columnModel.getColumnIndexAtX(p.x);  
            int realIndex = columnModel.getColumn(index).getModelIndex();  
            return columnToolTips[ realIndex ];  
        }  
    };  
}
```



The screenshot shows a window titled "TableToolTipsDemo" containing a table with 5 columns: First Name, Last Name, Sport, # of Years, and Vegetarian. The table contains 5 rows of data. A mouse cursor is hovering over the "Sport" column header, and a tooltip is displayed with the text "The person's favorite sport to participate in".

First Name	Last Name	Sport	# of Years	Vegetarian
Mary	Campione	Snowboarding	5	☐
Alison	Huml	Rowing	3	☒
Kathy	Walrath	Knitting	2	☐
Sharon	Zakhour	Speed reading	20	☒
Philip	Miles	Boat racing	40	☐



The screenshot shows the same "TableToolTipsDemo" window. A mouse cursor is now hovering over the "Rowing" cell in the "Sport" column of the second row. A tooltip is displayed with the text "This person's favorite sport to participate in is: Rowing".

First Name	Last Name	Sport	# of Years	Vegetarian
Mary	Campione	Snowboarding	5	☐
Alison	Huml	Rowing	3	☒
Kathy	Walrath	Knitting	2	☐
Sharon	Zakhour	Speed reading	20	☒
Philip	Miles	Boat racing	40	☐

Formatações: Tabelas

Interfaces Gráficos – Listagens e Tabelas

É possível criar ***tool tips*** para os **cabeçalhos** das colunas de uma tabela.

Para o efeito, temos que realizar uma implementação do método **`createDefaultTableHeader()`**, responsável pela definição do cabeçalho da tabela, retornando um objecto **`JTableHeader`**, com as propriedades do cabeçalho da tabela, incluindo os comportamentos que pretendemos incluir

Este método, por sua vez, deve incluir a implementação do método **`getToolTipText()`**, que recebe um evento do rato (**`MouseEvent`**, sendo possível extrair a posição do rato) e é responsável por retornar uma ***string*** com a ***tool tip*** para a posição actual do rato

Estas implementações podem ser realizadas através de métodos anónimos na declaração da tabela (**`JTable`**)

```
JTable table = new JTable(data, columnNames) {  
    protected JTableHeader createDefaultTableHeader() {  
        return new JTableHeader(columnModel) {  
            public String getToolTipText(MouseEvent e) {  
                // código para devolução da tooltip  
            }  
        }  
    };  
};
```

Formatações: Tabelas

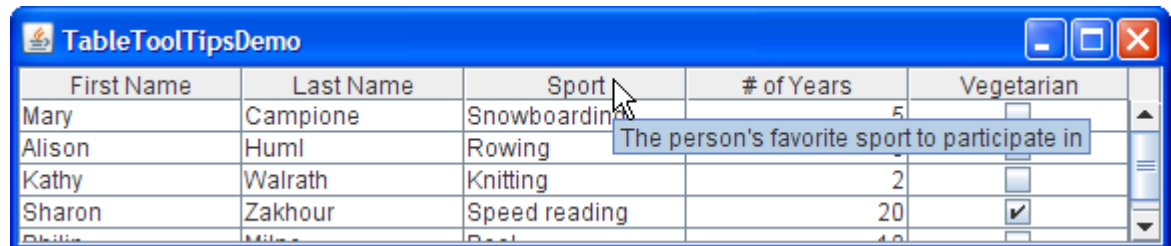
Interfaces Gráficos – Listagens e Tabelas

```
private String[] columnToolTips = {null,
                                   null,
                                   "The person's favorite sport to participate in",
                                   "The number of years the person has played the sport",
                                   "If checked, the person eats no meat"};

private String[] columnNames = {"First Name", "Last Name", "Sport", "# of Years", "Vegetarian"};

private Object[][] data = {
    {"Mary", "Campione", "Snowboarding", new Integer(5), new Boolean(false)},
    {"Alison", "Huml", "Rowing", new Integer(3), new Boolean(true)},
    {"Kathy", "Walrath", "Knitting", new Integer(2), new Boolean(false)},
    {"Sharon", "Zakhour", "Speed reading", new Integer(20), new Boolean(true)},
    {"Philip", "Milne", "Pool", new Integer(10), new Boolean(false)},
};

JTable table = new JTable(data, columnNames) {
    protected JTableHeader createDefaultTableHeader() {
        return new JTableHeader(columnModel) {
            public String getToolTipText(MouseEvent e) {
                java.awt.Point p = e.getPoint();
                int index = columnModel.getColumnIndexAtX(p.x);
                int realIndex = columnModel.getColumn(index).getModelIndex();
                return columnToolTips[realIndex];
            }
        };
    }
};
```



First Name	Last Name	Sport	# of Years	Vegetarian
Mary	Campione	Snowboarding	5	☐
Alison	Huml	Rowing	3	☒
Kathy	Walrath	Knitting	2	☐
Sharon	Zakhour	Speed reading	20	☒
Philip	Milne	Pool	10	☐

Formatações: Tabelas

Interfaces Gráficos – Listagens e Tabelas

Também é possível **criar *tool tips*** para as várias **células** de uma tabela

Tal como no caso anterior, na definição da tabela é necessário realizar a implementação de um método, sendo neste caso o **`getToolTipText()`**, similar ao processo das *tool tips* para o cabeçalho

Porém, como se pretende apenas interagir co células, não será necessário implementar o **`createDefaultTableHeader()`**, responsável pela gestão do cabeçalho (apenas caso se pretenda adicionalmente gerir as *tool tips* do cabeçalho)

```
JTable table = new JTable(data, columnNames) {  
    public String getToolTipText(MouseEvent e) {  
        java.awt.Point p = e.getPoint();  
        int rowIndex      = rowAtPoint(p);  
        int colIndex      = columnAtPoint(p);  
        return getValueAt(rowIndex, colIndex).toString();  
    }  
};
```

Nesta implementação, através do evento gerado pelo movimento do rato, obtém-se a posição do rato e deste a respectiva coluna e a linha da tabela, devolvendo, como exemplo, o texto da célula que será usado como *tool tip*

Formatações: Tabelas

Interfaces Gráficos – Listagens e Tabelas

```
private String[] columnToolTips = { ... }; // tabela com informação das tool tips para o cabeçalho
private String[] columnNames = { ... }; // tabela com nome dos cabeçalhos da JTable
private Object[][] data = { ... }; // tabela/vector de dados da Jtable

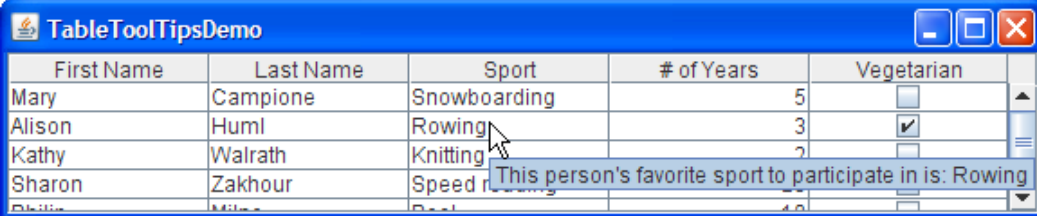
JTable table = new JTable(data, columnNames) {

    // método opcional, implementar caso se pretenda tooltip para o cabeçalho da tabela
    protected JTableHeader createDefaultTableHeader() {
        // Código
    }

    // método opcional, implementar caso se pretenda tooltip para cada célula da tabela
    public String getToolTipText(MouseEvent e) {
        String tip = null;
        java.awt.Point p = e.getPoint();
        int rowIndex = rowAtPoint(p);
        int colIndex = columnAtPoint(p);
        int realColumnIndex = convertColumnIndexToModel(colIndex);

        if (realColumnIndex == 2) { //Sport column
            tip = "This person's favorite sport to participate in is: " + getValueAt(rowIndex, colIndex);
        } else if (realColumnIndex == 4) { //Veggie column
            TableModel model = getModel();
            String firstName = (String)model.getValueAt(rowIndex,0);
            String lastName = (String)model.getValueAt(rowIndex,1);
            Boolean veggie = (Boolean)model.getValueAt(rowIndex,4);
            if (Boolean.TRUE.equals(veggie)) {
                tip = firstName + " " + lastName + " is a vegetarian";
            } else {
                tip = firstName + " " + lastName + " is not a vegetarian";
            }
        } else {
            tip = super.getToolTipText(e);
        }

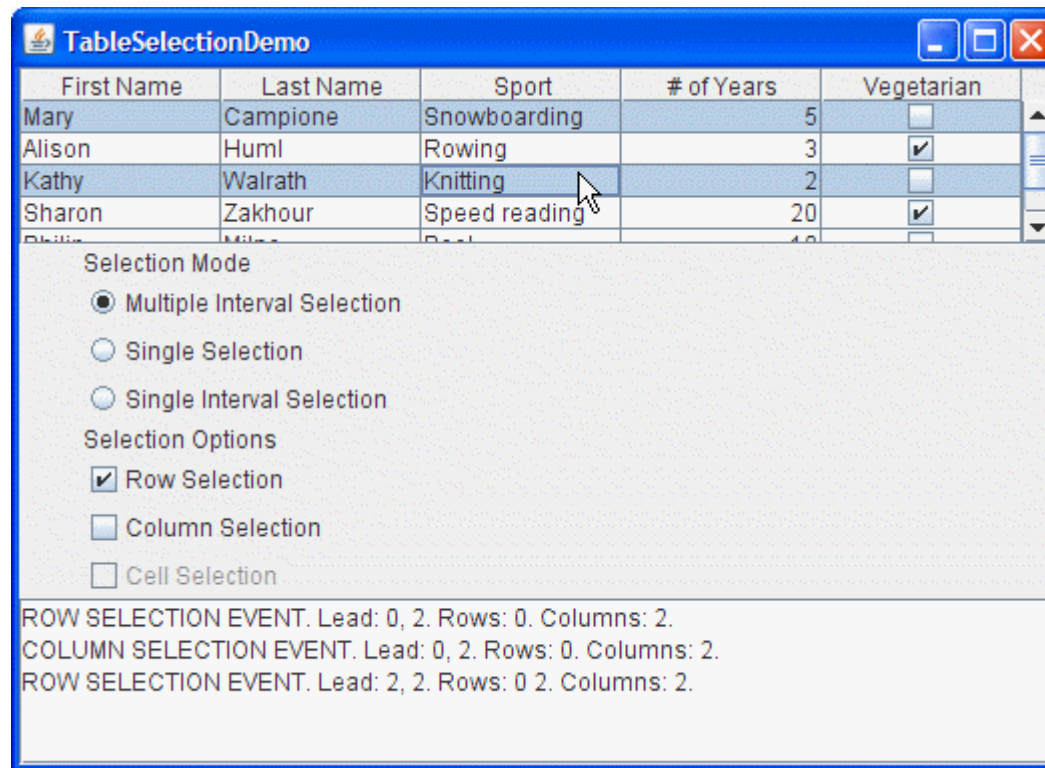
        return tip;
    }
};
```



First Name	Last Name	Sport	# of Years	Vegetarian
Mary	Campione	Snowboarding	5	☐
Alison	Huml	Rowing	3	☒
Kathy	Walrath	Knitting	2	☐
Sharon	Zakhour	Speed r...	1	☐
Philip	Miles	Base...	4	☐

Formatações: Tabelas

Interfaces Gráficas – Listagens e Tabelas



Mais informações em

<http://java.sun.com/docs/books/tutorial/uiswing/components/table.html>

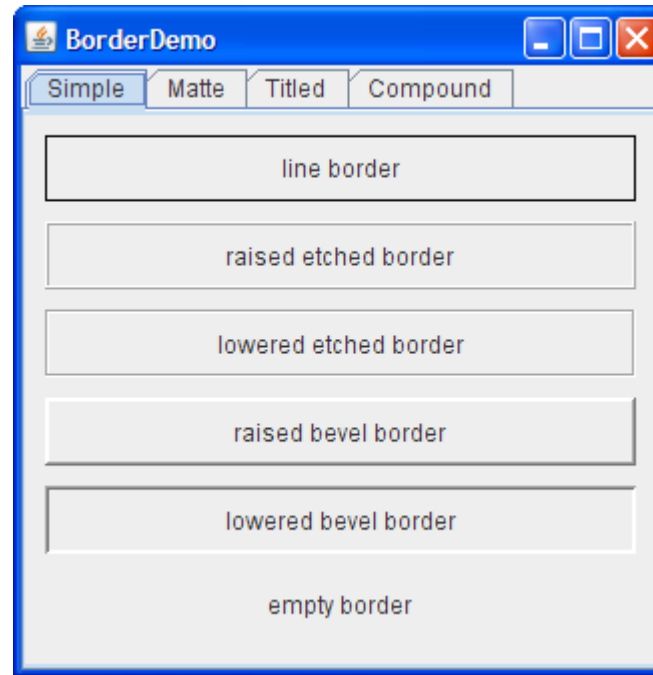
Organização de áreas:

**Margens, Abas, Barras de deslocamento e
Dimensionamento**

Margens

Formatações: Margens

Interfaces Gráficas – Listagens e Tabelas



Mais informações em

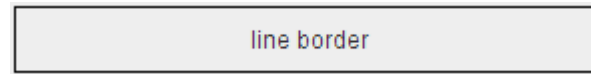
<http://java.sun.com/docs/books/tutorial/uiswing/components/border.html>

Formatações: Margens

Interfaces Gráficas – Listagens e Tabelas

É possível colocar **margens** envolvendo as diversas componentes, recorrendo ao método **setBorder()**, que recebe um objecto do tipo **BorderFactory** ou do tipo **TitledBorder**, entre outros

```
JPanel pane = new JPanel();  
pane.setBorder( BorderFactory.createLineBorder(Color.black) );
```



A classe **BorderLayout** não permite um controlo total sobre a margem, pelo que foram desenvolvidas várias implementações adicionais:

LineBorder,	EtchedBorder,	BevelBorder,	EmptyBorder,
MatteBorder,	TitledBorder,	CompoundBorder	SoftBevelBorder

A classe **LineBorder** permite definir a **cor**, **tamanho** e **arredondamento** (nos cantos) da margem:

```
JButton botao = new JButton("OK");  
botao.setBorder( new LineBorder(Color.RED, 6, false) );
```



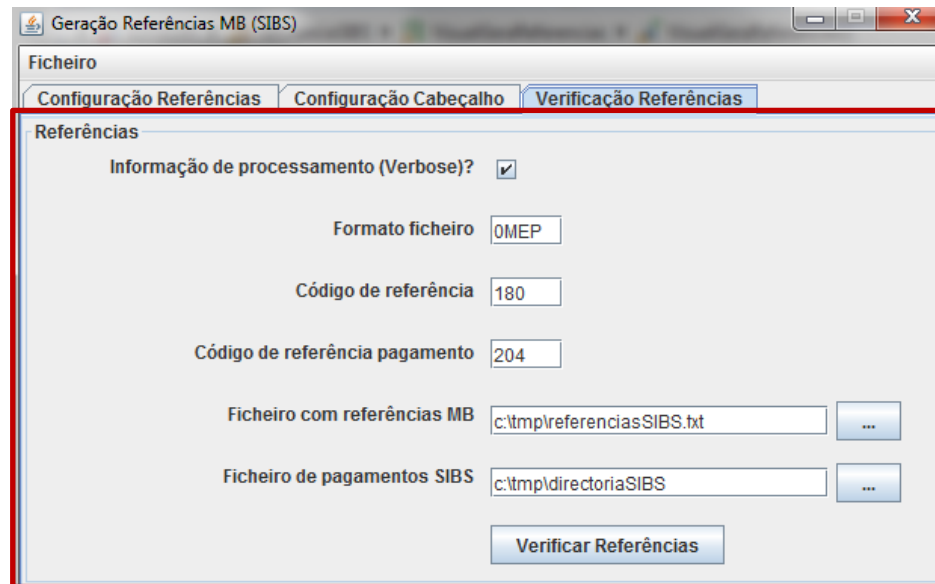
Formatações: Margens

Interfaces Gráficas – Listagens e Tabelas

As margens permitem assim delimitar áreas de trabalho, como em painéis com diversas componentes.

A classe **LineBorder** permite configurar a linha da margem enquanto **TitleBorder** permite adicionar um texto à margem

```
painelPrincipal = new JPanel( new GridLayout(7,2,1,1) );  
painelPrincipal.setBorder( new TitledBorder("Referências") );
```



Define a **cor**,
tamanho e
arredondamento
dos cantos

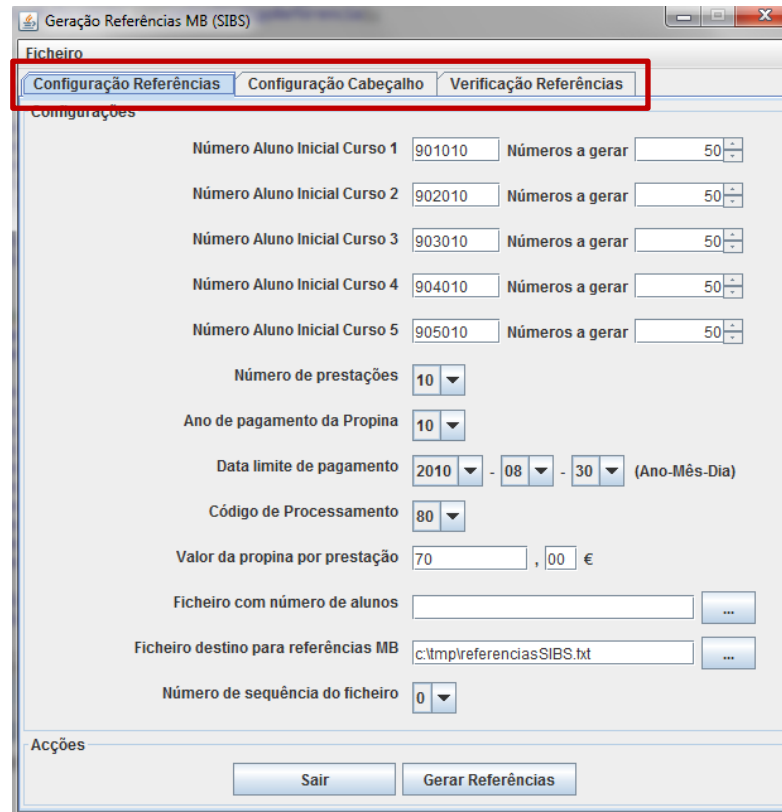
```
painelPrincipal.setBorder( new TitledBorder(  
    new LineBorder( Color.black, 1, false), // estilo da margem  
    "Referências" ) // título da margem  
);
```

Abas

Formatações: Abas

Interfaces Gráficas – Listagens e Tabelas

As **abas**, implementadas pela classe **JTabbedPane** são uma alternativa para navegar entre diferentes janelas ou painéis



Mais informações em

<http://java.sun.com/docs/books/tutorial/uiswing/components/tabbedpane.html>

Formatações: Abas

Interfaces Gráficas – Listagens e Tabelas

Para o efeito basta criar um objecto do tipo **JTabbedPane** e adicionar várias componentes (**JComponent**), por exemplo:

```
JTabbedPane tabbedPane = new JTabbedPane();  
ImageIcon icon = createImageIcon("images/middle.gif");  
  
JComponent panel1 = makeTextPanel("Panel #1");  
tabbedPane.addTab("Tab 1", icon, panel1, "Comment 1");  
  
JComponent panel2 = makeTextPanel("Panel #2");  
tabbedPane.addTab("Tab 2", icon, panel2, "Comment 2");
```



Estas componentes podem ser **JFrames**, **JPanels** ou outros elementos (**JComponent**)

Mais informações em

<http://java.sun.com/docs/books/tutorial/uiswing/components/tabbedpane.html>

Formatações: Abas

Interfaces Gráficos – Listagens e Tabelas

```
JTabbedPane tabbedPane = new JTabbedPane();  
ImageIcon icon = createImageIcon("images/middle.gif");  
  
JComponent panel1 = makeTextPanel("Panel #1");  
tabbedPane.addTab("Tab 1", icon, panel1, "Does nothing");  
tabbedPane.setMnemonicAt(0, KeyEvent.VK_1);  
  
JComponent panel2 = makeTextPanel("Panel #2");  
tabbedPane.addTab("Tab 2", icon, panel2, "Does twice as much nothing");  
tabbedPane.setMnemonicAt(1, KeyEvent.VK_2);  
  
JComponent panel3 = makeTextPanel("Panel #3");  
tabbedPane.addTab("Tab 3", icon, panel3, "Still does nothing");  
tabbedPane.setMnemonicAt(2, KeyEvent.VK_3);  
  
JComponent panel4 = makeTextPanel("Panel #4 (Size of 410 x 50).");  
panel4.setPreferredSize(new Dimension(410, 50));  
tabbedPane.addTab("Tab 4", icon, panel4, "Does nothing at all");  
tabbedPane.setMnemonicAt(3, KeyEvent.VK_4);  
  
add(tabbedPane);
```

Barras de Deslocamento

Formatações: Componentes com deslocamento

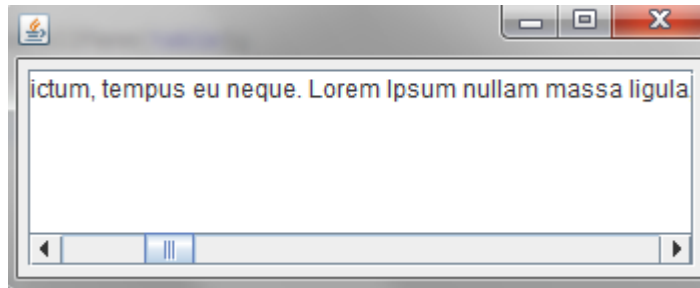
Interfaces Gráficas – Listagens e Tabelas

Componentes com deslocamento permitem criar componentes com barras de deslocamento (**JScrollPane**)

O efeito pretendido é obtido adicionado uma componente a um objecto do tipo JScrollPane

```
JTextArea textArea = new JTextArea(5, 30);  
textArea.setText("Lorem Ipsum ...");
```

```
JScrollPane scrollPane = new JScrollPane( textArea );  
add( scrollPane );
```



Mais informações em

<http://java.sun.com/docs/books/tutorial/uiswing/components/scrollpane.html>

Formatações: Barra de deslocamento

Interfaces Gráficos – Listagens e Tabelas

As **barras de deslocamento** (`ScrollBar`), disponíveis, por exemplo, nas componentes `JScrollPane` permitem navegar através de uma componente

É possível **definir a posição da barra horizontal e vertical** através dos métodos

```
setHorizontalScrollBar()
```

```
setVerticalScrollBar()
```

Também é possível **obter a sua posição** através dos métodos

```
getHorizontalScrollBar()
```

```
getVerticalScrollBar()
```

Formatações: Barra de deslocamento

Interfaces Gráficas – Listagens e Tabelas

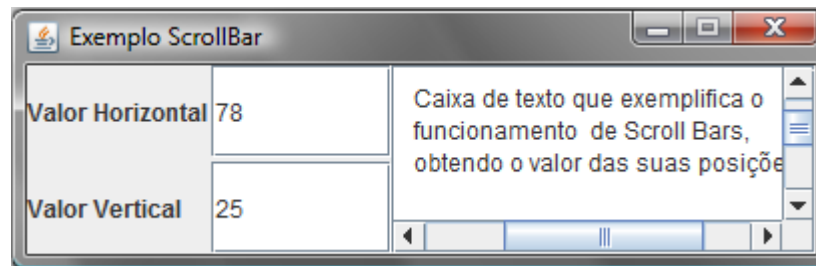
Para obtermos os **eventos associados ao movimento das barras** recorremos ao *listener*

AdjustmentListener

Este *listener* (ou gestor de eventos) implica a **implementação do método**

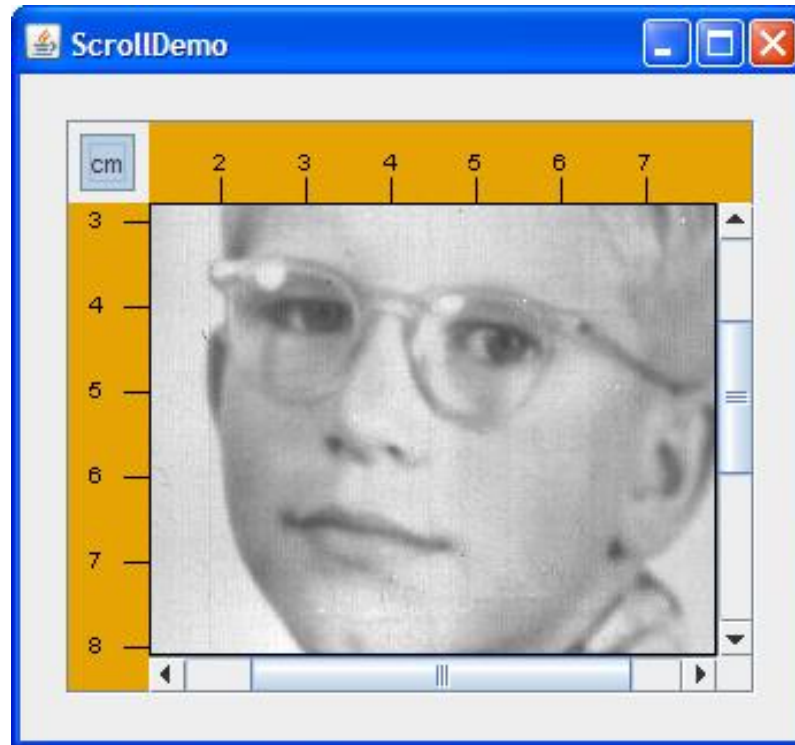
`adjustmentValueChanged(AdjustmentEvent ae)`

```
JTextArea area = new JTextArea("Texto", 5, 15);  
JScrollPane scrollPane = new JScrollPane(area);  
scrollPane.getHorizontalScrollBar().addAdjustmentListener(this);  
scrollPane.getVerticalScrollBar().addAdjustmentListener(this);
```



Formatações: Componentes com deslocamento

Interfaces Gráficas – Listagens e Tabelas



Dimensões e Focus

Formatações: Dimensão

Interfaces Gráficos – Listagens e Tabelas

É possível **definir o tamanho de uma componente**, recorrendo ao método **`setPreferredSize()`** (não confundir com o método **`setSize()`**)

O método recebe como argumento um objecto do tipo **`Dimension`** (que **define o comprimento e a altura** da componente):

```
JButton botao = new JButton("OK");  
botao.setPreferredSize( new Dimension (100, 20 ));
```

Após o utilizador alterar o tamanho das componentes é possível **repor o tamanho original** através do método

```
resetToPreferredSize()
```


De realçar que também existe o método `setSize()`, porém a sua instrução **não se sobrepõem ao redimensionamento que alguns gestores de posicionamento forçam**, como o caso do `GridLayout`

Algumas componentes permitem **definir o tamanho mínimo e máximo da componente** (útil em situações em que o utilizador pode redimensionar a janela) através dos métodos

`setMinimumSize()` e

`setMaximumSize()`

Para **invocar o *focus*** da aplicação sobre uma componente (elemento que ficará sujeito a acção por defeito, por exemplo uma caixa de texto num formulário) é possível recorrer a um dos seguintes métodos

`requestFocus()` OU

`requestFocusInWindow()`

Podemos também invocar o *focus* sobre uma janela através do método `toFront()`; ou garantir mesmo que esta se sobrepõe a todas as aplicações visíveis ecrã através do método

`setAlwaysOnTop(true)` ;

Também é possível **definir a acção por defeito** quando uma tecla é pressionada, implementando a interface `KeyListener` e verificando o código da tecla pressionada no método de gestão de eventos `keyPressed()`:

```
public void keyPressed(KeyEvent e) {  
    if (e.getKeyCode() == KeyEvent.VK_ENTER) { // ENTER  
        buttonConfirmation.doClick();  
    }  
}
```

Formatação de campos alfanuméricos:

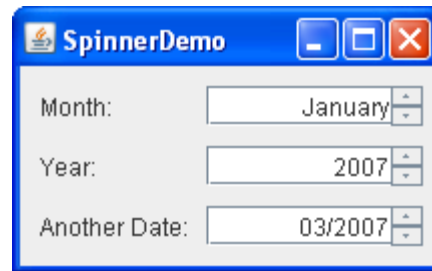
Spinners, Campos de texto formatados e Formatação de texto, números e data

Spinners

Formatações: JSpinner

Interfaces Gráficos – Listagens e Tabelas

As componentes designadas como ***spinners*** (implementadas pela classe `JSpinner`) são semelhantes a *combo box* ou listas, **permitindo que o utilizador escolha uma das opções disponíveis a partir de um grupo**



Mais informações em

<http://java.sun.com/docs/books/tutorial/uiswing/components/spinner.html>

Formatações: JSpinner

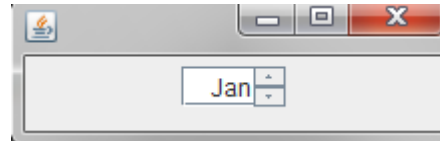
Interfaces Gráficos – Listagens e Tabelas

Para criar um *spinner* :

- 1 - primeiro é necessário criar um modelo (**SpinnerListModel**)
- 2 - e só depois associa-lo ao *spinner* (**JSpinner**)

```
String[] months = {"Jan", "Fev", ..., "Nov", "Dez"}
```

```
SpinnerListModel monthModel = new SpinnerListModel( months );  
JSpinner spinner = new JSpinner( monthModel );  
add( spinner );
```

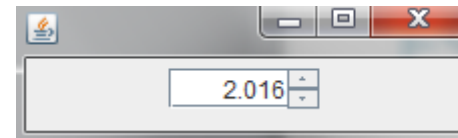
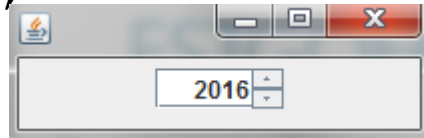


Formatações: JSpinner

Interfaces Gráficas – Listagens e Tabelas

O modelo **SpinnerModel** suporta e gere sequências de números, sendo necessário definir o valor inicial, máximo, mínimo e de cada incremento

```
int currentYear = 2016;  
SpinnerModel model = new SpinnerNumberModel(  
    currentYear,           //valor inicial  
    currentYear - 100,     //valor mínimo  
    ,                      //valor máximo  
    1currentYear + 100  
    //incremento  
);  
JSpinner spinner = new JSpinner( model );  
spinner.setEditor( new JSpinner.NumberEditor(spinner, "#") );  
add(spinner);
```



O método **setEditor()** permite alterar a visualização dos dados do *spinner*, enquanto o método **NumberEditor** permite formatar a apresentação de algarismos

Formatações: JSpinner

Interfaces Gráficas – Listagens e Tabelas

Podemos **alterar a formatação dos número e datas** através das classes

`JSpinner.NumberEditor` e

`JSpinner.DateEditor`

Exemplo:

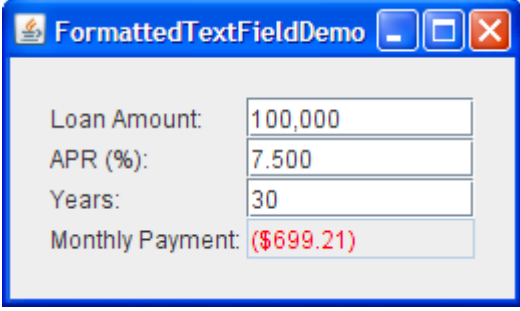
```
spinner.setEditor(  
    new JSpinner.DateEditor(spinner, "MM/yyyy")  
);
```

Uma alteração ao valor do *spinner* gera um **evento do tipo `ChangeListener`**, estando o seu tratamento a cargo do método `stateChanged()`

Campos de Texto Formatados

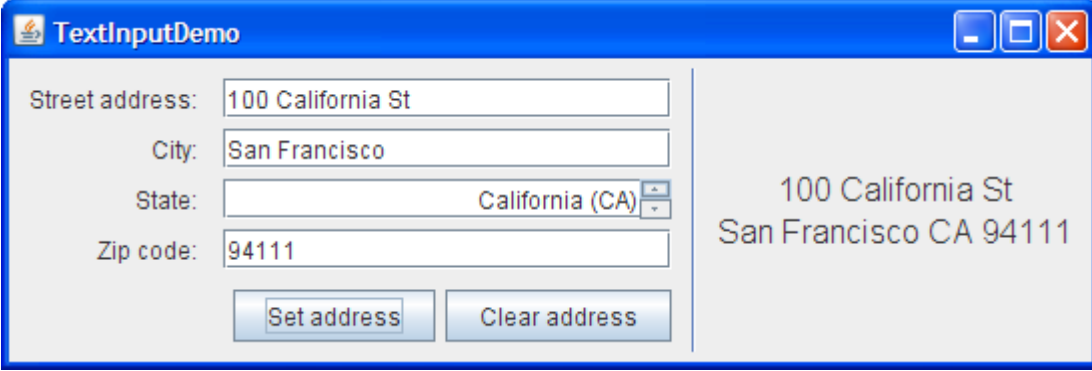
Formatações: Campos de texto formatados

Interfaces Gráficas – Listagens e Tabelas



A screenshot of a Java Swing window titled "FormattedTextFieldDemo". It contains four text fields with the following labels and values:

Label	Value
Loan Amount:	100,000
APR (%):	7.500
Years:	30
Monthly Payment:	(\$699.21)



A screenshot of a Java Swing window titled "TextInputDemo". It contains four text fields for address information, a state dropdown menu, and two buttons. The values are as follows:

Field	Value
Street address:	100 California St
City:	San Francisco
State:	California (CA)
Zip code:	94111

Buttons: "Set address", "Clear address"

Summary text on the right: "100 California St
San Francisco CA 94111"

Mais informações em

<http://java.sun.com/docs/books/tutorial/uiswing/components/spinner.html>

Formatações: Campos de texto formatados

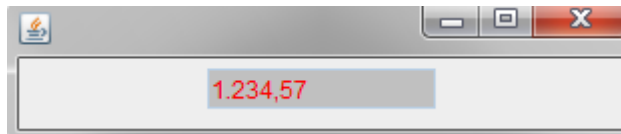
Interfaces Gráficas – Listagens e Tabelas

Em alternativa aos campos de texto tradicionais (`JTextField`) é possível criar **campos de texto que permitem a formatação do seu conteúdo** recorrendo à classe `JFormattedTextField` e aos diversos métodos de formatação:

```
double amount = 1234.567;

NumberFormat amountFormat = NumberFormat.getNumberInstance();
amountFormat.setMinimumFractionDigits(2);
amountFormat.setMaximumFractionDigits(2);

JFormattedTextField amountField
    = new JFormattedTextField(amountFormat);
amountField.setValue(amount);
amountField.setColumns(10);
amountField.setEchoChar((char)0);
amountField.setForeground(Color.red);
amountField.setBackground(Color.lightGray);
amountField.addPropertyChangeListener("value", this)
add(amountField);
```



De realçar que um campo de texto formatado e os seus valores são duas propriedades diferentes

Formatações: Campos de texto formatados

Interfaces Gráficos – Listagens e Tabelas

A **formatação de números**, com incidência nos números reais (tipo *float* ou *double*) é possível através da classe **NumberFormat**

Para o efeito é criado um objecto do tipo `NumberFormat` que define a estrutura:

```
NumberFormat formatter = new DecimalFormat("0.##");
```

No caso anterior o número apresentará duas casas decimais (.##).

O símbolo **0** apresenta o dígito zero, caso nenhum número seja presente, enquanto o símbolo **#** só apresenta o respectivo dígito se o número estiver presente. O símbolo **.** representa o ponto decimal

A seguir aplica-se esse formato ao número pretendido:

```
String s = formatter.format(-1234.567);
```

Formatações: Campos de texto formatados

Interfaces Gráficas – Listagens e Tabelas

```
// O símbolo 0 apresenta o dígito 0 se este não estiver presente
NumberFormat formatter = new DecimalFormat("000000");
String s = formatter.format(-1234.567); // -001235 (número foi arredondado)

// O símbolo # não apresenta o dígito caso o número não esteja presente
formatter = new DecimalFormat("###");
s = formatter.format(-1234.567); // -1235
s = formatter.format(0); // 0
formatter = new DecimalFormat("##00");
s = formatter.format(0); // 00

// O símbolo . indica o ponto decimal
formatter = new DecimalFormat(".00");
s = formatter.format(-.567); // -.57
formatter = new DecimalFormat("0.00");
s = formatter.format(-.567); // -0.57
formatter = new DecimalFormat("#.#");
s = formatter.format(-1234.567); // -1234.6
formatter = new DecimalFormat("#.#####");
s = formatter.format(-1234.567); // -1234.567
formatter = new DecimalFormat(".#####");
s = formatter.format(-1234.567); // -1234.567
formatter = new DecimalFormat("#.000000");
s = formatter.format(-1234.567); // -1234.567000
```

Formatações: Campos de texto formatados

Interfaces Gráficos – Listagens e Tabelas

// O símbolo , é usado para agrupar números

```
formatter = new DecimalFormat("#,###,###");
```

```
s = formatter.format(-1234.567);           // -1,235
```

```
s = formatter.format(-1234567.890);        // -1,234,568
```

*// O símbolo ; é usado para especificar um padrão alternativo para valores
// negativos*

```
formatter = new DecimalFormat("#;(#)");
```

```
s = formatter.format(-1234.567);           // (1235)
```

// O símbolo ' é usado para indicar símbolos literais

```
formatter = new DecimalFormat("'#'#");
```

```
s = formatter.format(-1234.567);           // -#1235
```

```
formatter = new DecimalFormat("'abc'#");
```

```
s = formatter.format(-1234.567);           // -abc1235
```

Formatações: Limitação de caracteres em campos de texto

Interfaces Gráficas – Listagens e Tabelas

É possível restringir o número de caracteres visíveis em cada campo de texto através de validação

A primeira abordagem implica **associar um evento `ActionListener`** ao campo de texto, **verificando o comprimento do texto sempre que o utilizador pressiona uma tecla**, mantendo o número de caracteres limitado

Uma abordagem alternativa baseia-se no uso da propriedade **`setDocument()`** que implica implementar uma validação através da interface **`DocumentFilter`** OU **`PlainDocument`**

Por fim, é possível **reescrever a classe `JTextField`** para adicionar um parâmetro adicional (comprimento máximo), controlado através de uma das interfaces **`DocumentFilter`** OU **`PlainDocument`**

Formatações: Limitação de caracteres em campos de texto

Interfaces Gráficos – Listagens e Tabelas

```
import javax.swing.text.PlainDocument;

public class JTextFieldLimit extends PlainDocument {
    private int limit;

    JTextFieldLimit(int limit) {
        super();
        this.limit = limit;
    }

    // método responsável pela validação
    public void insertString( int offset, String str, AttributeSet attr )
        throws BadLocationException {
        if (str == null) return;
        if ((getLength() + str.length()) <= limit) {
            super.insertString(offset, str, attr);
        }
    }
}
```

Tendo sido criada a classe responsável, podemos associar essa propriedade à tua caixa de texto:

```
// tamanho visual da caixa de texto (número de colunas)
JTextField textfield = new JTextField(10);
textfield.setDocument(new JTextFieldLimit(5));
// número máximo de caracteres, usando a classe criada anteriormente (JTextFieldLimit) para
esse efeito.
```

Formatações: Limitação de caracteres em campos de texto

Interfaces Gráficos – Listagens e Tabelas

Implementação alternativa usando a interface **DocumentFilter**:

```
public class LengthFilter extends DocumentFilter {
    private int max;

    public LengthFilter(int maxLength) {
        max = maxLength;
    }

    public void insertString(DocumentFilter.FilterBypass fb, int offset,
                             String text, AttributeSet attr) throws BadLocationException {
        if (fb.getDocument().getLength() + text.length() <= max)
            fb.insertString(offset, text, attr);
        else
            Toolkit.getDefaultToolkit().beep();
    }

    public void replace(DocumentFilter.FilterBypass fb, int offset,
                        int length, String text, AttributeSet attr) throws BadLocationException {
        if (fb.getDocument().getLength() + text.length() - length <= max)
            fb.replace(offset, length, text, attr);
        else
            Toolkit.getDefaultToolkit().beep();
    }
}
```

Formatações: Limitação de caracteres em campos de texto

Interfaces Gráficas – Listagens e Tabelas

Alternativamente é possível reescrever o próprio objecto da caixa de texto (mas recorrendo sempre a uma das interfaces anteriores):

```
public class LimitField extends JTextField {
    private int maximum ;

    public LimitField( int columns, int max ) {
        super( columns );
        maximum = max ;
    }

    public LimitField( String text, int columns, int max ) {
        super( text, columns );
        maximum = max;
    }

    protected Document createDefaultModel() {
        return new LimitDocument();
    }

    private class LimitDocument extends PlainDocument {
        public void insertString(int offs, String str, AttributeSet a) throws BadLocationException {
            StringBuffer buffer = new StringBuffer( getText( 0, getLength() ) );
            if ( ( buffer.length() + str.length() ) <= maximum ) {
                super.insertString(offs, str, a);
            } else {
                Toolkit.getDefaultToolkit().beep();
            }
        }
    }
}
```

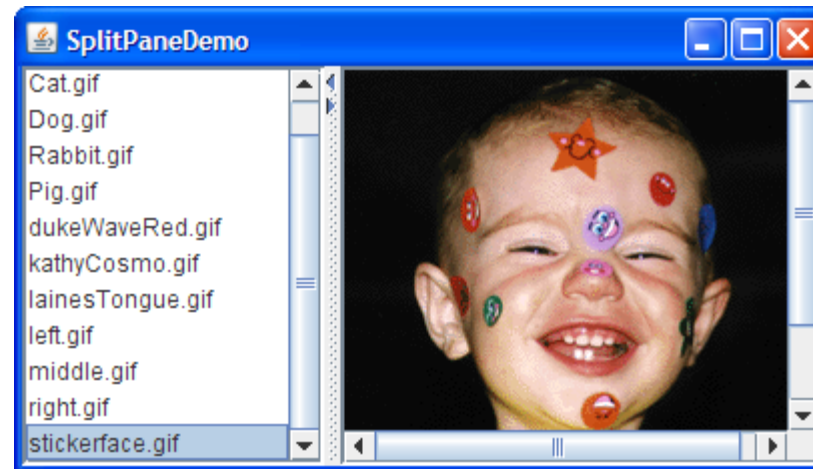
Organização de áreas: Divisão de Áreas e Árvores

Divisões de Área

Formatações: Divisão de áreas

Interfaces Gráficas – Listagens e Tabelas

De realçar que cada área pode, por sua vez, ser subdivida em `JSplitPane`



Mais informações em

<http://java.sun.com/docs/books/tutorial/uiswing/components/splitpane.html>

Formatações: Divisão de áreas

Interfaces Gráficos – Listagens e Tabelas

A **divisão de áreas** por ser conseguido através da componente **JSplitPane**

Esta componente apresenta duas áreas vizinhas (na horizontal ou vertical, através das variáveis **JSplitPane.HORIZONTAL_SPLIT** e **JSplitPane.VERTICAL_SPLIT** definidas no construtor ou através do método **setOrientation()**)

O utilizador pode definir o tamanho que cada área vizinha ocupa arrastando o separador central

```
splitPane = new JSplitPane(JSplitPane.HORIZONTAL_SPLIT,  
                           listScrollPane, pictureScrollPane);  
splitPane.setOneTouchExpandable(true);  
splitPane.setDividerLocation(150);
```

Formatações: Divisão de áreas

Interfaces Gráficos – Listagens e Tabelas

O método **`setOneTouchExpandable()`** permite expandir uma área ocupando todo o espaço da janela clicando numa seta do separador central

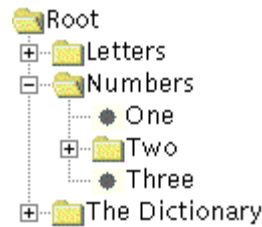
O método **`setDividerLocation()`** define o ponto (em pixels) de separação de ambas as áreas, sendo possível definir o ponto de separação através do peso que cada área representará recorrendo ao método `setResizeWeight()`, que recebe valores entre 0 e 1 (para uma separação equilibrada cada área deverá apresentar um peso de 0.5)

Árvores (Jtree)

Formatações: Árvores

Interfaces Gráficas – Listagens e Tabelas

As **árvores** (**JTree**) **permitem apresentar a informação de uma forma hierárquica**, embora não contenham dados, apenas permitem a sua visualização



Cada registo corresponde a um nó (*node*, classe **DefaultMutableTreeNode**),

o primeiro registo a raiz principal (*root*, através do método **createNodes()**) de onde derivam todos os restantes registos.

Cada nó pode apresentar nós ‘filhos’ (*branch*), ou encontra-se isolado, definindo-se neste caso como nó folha (*leaf*)

É possível detectar alterações no estado de expansão dos nós ‘filhos’ implementado os *listeners*

TreeWillExpandListener e

TreeExpansionListener

Criação da estrutura principal:

```
private JTree tree;    // objecto principal da árvore
...
public TreeDemo() {
    ...
    // criação de um nó (DefaultMutableTreeNode)
    DefaultMutableTreeNode top =
        new DefaultMutableTreeNode("The Java Series");
    // definição do nó raiz
    createNodes(top);
    // criação e inicialização da árvore com o nó raiz
    tree = new JTree(top);
    ...
    // adicionar a árvore a um scrollpane para permitir deslocamento
    JScrollPane treeView = new JScrollPane(tree);
    ...
}
```

Formatações: Árvores

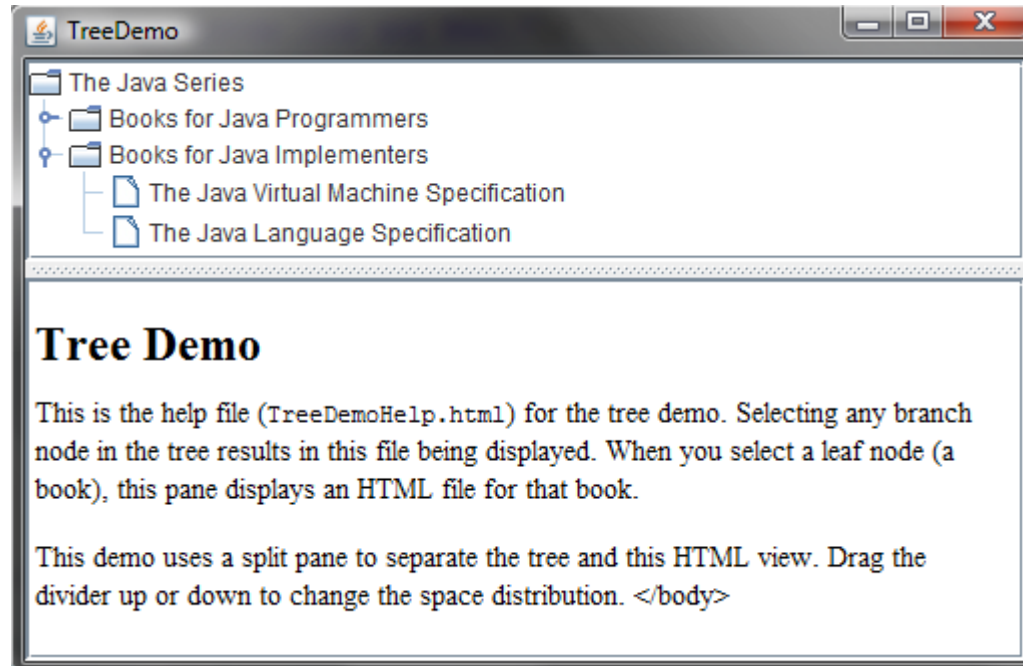
Interfaces Gráficos – Listagens e Tabelas

Criação da estrutura principal:

```
private void createNodes(DefaultMutableTreeNode top) {  
    DefaultMutableTreeNode category = null;  
    DefaultMutableTreeNode book = null;  
  
    // book category  
    category = new DefaultMutableTreeNode("Books for Java Programmers");  
    top.add(category);    // nó adicionado à raiz  
  
    //book: original Tutorial  
    book = new DefaultMutableTreeNode(new BookInfo  
        ("The Java Tutorial: A Short Course on the Basics",  
         "tutorial.html") );  
    category.add(book);    // nó folha adicionado a um nó superior  
  
    //book: Tutorial Continued  
    book = new DefaultMutableTreeNode(new BookInfo  
        ("The Java Tutorial Continued: The Rest of the JDK",  
         "tutorialcont.html") );  
    category.add(book);  
  
    //book: JFC Swing Tutorial  
    book = new DefaultMutableTreeNode(new BookInfo  
        ("The JFC Swing Tutorial: A Guide to Constructing GUIs",  
         "swingtutorial.html") );  
    category.add(book);  
    ...  
};
```

Formatações: Divisão de áreas e Árvores

Interfaces Gráficas – Listagens e Tabelas



Mais informações em

<http://java.sun.com/docs/books/tutorial/uiswing/components/tree.html>

“Programação Orientada a Objectos”

António José Mendes

Departamento de Engenharia Informática, Universidade de Coimbra

“Java Swing”, 2ª Edição

James Elliott, Robert Eckstein (Editor), Marc Loy, David Wood, Brian Cole
O'Reilly, ISBN: 0596004087

“Thinking in Java, ”, 4ª Edição, Capítulo 14 “Creating Windows & Applets”

Bruce Eckel

Prentice Hall, ISBN: 0131872486

“The Java Tutorial – Creating a GUI with JFC/Swing”

Java Sun Microsystems

<http://java.sun.com/docs/books/tutorial/uiswing/index.html>

Referências

Interfaces Gráficos – Listagens e Tabelas

“The Definitive Guide to Java Swing”, 3ª Edição

John Zukowski

Apress, ISBN: 1590594479

“Fundamentos de Programação em Java 2”, Capítulo 12 *“Interfaces Gráficos”*

António José Mendes, Maria José Marcelino

FCA, ISBN: 9727224237